

Naming conventions	
Meaningful names for variables	C001
True to intention , no interpretation	
Maximum length for names	C002
Better readability → 24 characters	
Avoid using abbreviations	C003
Abbreviations used should be documented.	
Don't reuse variable names	C004
Same names make it difficult to debug	
Define the use of case	C005
Constants → UPPER_SNAKE_CASE Multi-word items → UpperCamelCase	
Prefixes and suffixes	C006 C007
Prefix → g, di, do, ai, ao Suffix → Type (StatusType), Enum (StateEnum)	
Names of functions and function block (FB) instances	C008
Function: CalculateAverageValue FB instance : TuneTemperatureControllerZone1	
Don't repeat context name	C009
User defined types NO : Conveyor.Cmd.CmdStart YES : Conveyor.Cmd.Start	
Limit nesting of structures	C010
Recommended maximum of 4 levels nesting YES: Heater.Par.PID.Gain	

Readability	
Indentation	C101
Indent code with one Tab (= 4 spaces)	
Whitespaces (□ = one whitespace)	C102
Operators : Addend1□+□Addend2 Parenthesis : ADR('Recipelcecream') Assignment : RecipeXml.Enable□:=□TRUE; Function call arguments : Sum(Addend1,□Addend2)	
Separation of logical blocks visually	C103
Consistent empty lines between blocks.	
Define the maximum width of code	C104
Define number of characters to be used in a single line of code. Suggestion 120 characters	
Define the maximum length of code	C105
Set a limit for the maximum number of lines for a block of code.. Goal: avoid scrolling.	
Parentheses → Consistent	C106

Comments („Why“ not „How“)	
Describe all elements	C201
Programs, packages, libraries, configurations variables, user types, enumerations	
Consistent style and placement	C202
//comment instead of (* comment *) Place comments above code.	
Header	C203
File name, copyright, author, created on, description	
Don't comment out code	C203

Coding practices	
Initialize variables	C301
Delete unused variables and datatypes	C302
Global variables	C303 C304
Limited use only; Assign values within one program	
Function block instances	C305
Call only once per program cycle	
Delete dead code	C306
Avoid duplicating code	C307
Don't Repeat Yourself, use functions, FBs, actions, etc.	
Avoid implicit type conversions	C308
Avoid comparing floating points	C309
Limit program complexity	C310
Nested IF statements → maximum 4 levels	
Shorten boolean assignments	C311
NO : IF PendingAlarms = 0 THEN AlarmActive:= FALSE; ELSE AlarmActive:= TRUE; END_IF; YES : AlarmActive := (PendingAlarms > 0);	
Avoid using EDGE, EDGEPOS and EDGENEG	C312
Release code with 0 errors and 0 warnings	C313
No magic numbers. Define constants.	C314
Use AND instead of nested IF/ELSE	C315
Avoid boolean comparisons in conditions	C316
NO : IF Enable = TRUE THEN YES : IF Enable THEN	
Use structures to group inputs and outputs	C317